NOTEBOOK

Jenkins-Devops

Sarthak Dutta

Jenkins Fundamentals. pgs Estrev.

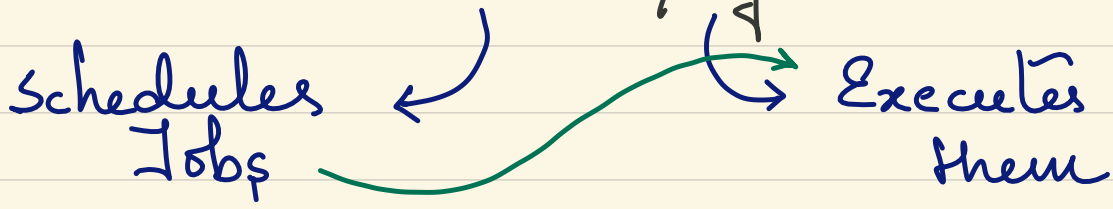
No.	Schedule	Description	Time
1	What is Jenkins ?	3	5 min
2	Why use Jenkins ?	4	5min
3	Workflow diagram.	4	5min
4	Architecture of Jenkins	5	10 min
5	Installing Jenkins.	6	--
6	Job and Projects	6-8	15min
7	Sample Practice	8	--
8	Credentials	9	10min
9	Build, Workspace, Console	10-11	15min
10	Executors & Nodes	12	10 min
11			
12			
13			
14			
15			
16			
17			
18			
19			
20			
21			
22			
23			
24			
25			

StarNote

1. What is Jenkins?

Open Source, automation server, written in Java. It uses pipeline to build, test and deploy whenever an event is triggered such as code commit, a schedule or a manual click.

It runs on Controller/agent architecture:



has an wide range of plugins which connects → Git, Docker, Kubernetes, testing

2. Why use Jenkins?

- Continuous Integration: Every code push is Automatically build & tested. → Integration Bug are caught early
- Continuous Delivery/Deployment: Same pipeline packages the apps & push it to → production & Staging with manual testing.

StarNote

- **Automation & Consistency**: Repeatable pipeline replaces error-prone manual builds & deployments.

- **Faster feedback**: Devs get build & test within minutes.

- **Flexibility**: Plugins allows integration with almost every toolchain and can run distributed builds across many

(3.) developer pushes code → Jenkins pulls it → Compile → runs unit tests → Builds a docker img → deploys to a test environment → notifies team.

popular alternative: GitHub Actions.

StarNote

(4) Jenkins Architecture

uses controller-agent formerly (master-slave) distributed architecture.

Controller (master): Hosts the Web UI, Stores configuration and job definitions, schedules builds, dispatches work to agent and records.

It can run builds itself, but in production it is used for orchestration only.

Agent (Slaves/Nodes): Machines (physical VMs, Container, or cloud instances) that actually executes the build steps. They connect to the controller via SSH/Websocket

Executors: Slots on a node that run jobs. A node with 4 executors can run 4 builds in parallel.

Jobs/pipelines: defined either as freestyle projects or as pipeline code in

(*) Why use agent instead of Controller
- memory exhaustion & overloading

instead of doing 500 task via 1 master divide 50 tasks per agent total 10 agent (load-balancing)

5. Installing Jenkins & using Jenkins

Sudo dnf install -y java-21-amazon-corretto.

Sudo wget -O /etc/yum....
Sudo rpm --import https://pkg....

Sudo dnf install -y jenkins

6. Job's & Projects

In Jenkins, a job (also called a project) configured unit of work that Jenkins runs

- → what to do
- → when to trigger
- → what to do with results

Each execution of a job is called a build.

StarNote

Main Job types:

What Job Configures:

- Source code management: Git repo & branch
- Triggers: webhook, poll SCM, cron schedule
- Build env: parameters, credentials.
- Build steps: Compile, test, package, deploy
- Post-build: archive artifacts, publish report

a) Freestyle project:

GUI Configured Job: fill in forms for generic Job Configs. in UI and config as XML, Easy for Simple tasking and no coding, No parallelism or stages hard to manage version control

b) Pipeline: Groovy-based Syntax code

for defining stages, parallel steps & long-running flows. written in (Jenkins file.)

Declarative: Structured and Simpler with fixed layout.

Scripted: Fully Groovy, more flexible but more complex.

StarNote

Key features of Pipeline over freestyle? Pg-8

Parallel: Execution: Run tests on multiple environment at the same time.

Durable: Survives controller restarts.

Version Controlled: Changes are reviewed and tracked in Git.

7. Sample practice. (UI & Dashboard)

StarNote

(8.) Credential Management

Credentials in Jenkins are Secrets (passwords, tokens, keys) stored securely by the controller → Jobs can use them without hardcoding them in <Jenkinsfile>

Common credentials:

- Username with password.
- Secret text
- SSH username with pvt key.
- Secret file.
- Certificate & Aws credentials.

Scopes:

- Global: usable by any Job and the system.
- System: Only for Jenkins (agents & email), not exposed to Jobs.
- Folder level: Restricted to Jobs inside specific folder.

StarNote

(9.) Build : Single execution of a job, Each 'run' gets a unique sequential number (#1, #2, #3....) and a status:

Success [● ●], Failed [●], Unstable [●], Aborted [●].

Jenkins stores each build's logs, artifacts, test results and metadata under Jenkins_Home/jobs/<job>/builds/<number> with a provision to auto-delete via 'discard old builds'.

Workspace : Directory on the node where Jenkins runs a job. It's where source code checked out, files are compiled & temporary build output is created. (C&A)

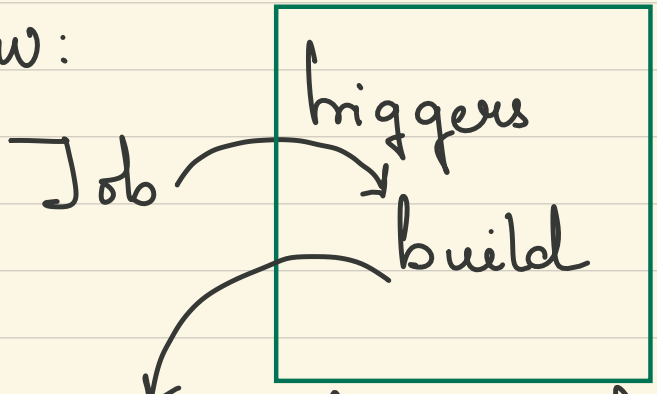
- Default Path: /var/lib/Jenkins/workspace/<job-name>
- persists between the builds of Job unless we clean it. cmd: cleanWs() in a pipeline.
- Each job gets its own workspace and concurrent builds get separate ones.

Console Output: live, line by line log of a build, showing everything Jenkins did:

- SCM checkout
- Shell command
- errors
- output
- plugin messages
- final result.

helps in debugging and also shows timestamps if enabled.

Workflow:



Remote directory: folder on agent where workspace is created.

StarNote

(10.) Node & Executors.

Node : Any machine that can run Jenkins builds.

- Controller (built-in node):

Machine Jenkins runs itself on. Set executor to 0 ⟶ only orchestrates!

- Agent Node: Separate machine (physical servers, VM, Docker containers, kubernetes pods, Instances) that run builds.

Executors : A slot on a node that runs one build at a time

Node → 3E → run 3 builds in parallel.

[Extra builds wait on build queue until executor is free.]

Components :

- Executor count is usually set near node's CPU core count.
- Labels : Tag like "linux, docker or windows" lets job target specific Node.
- Usage Settings : "Use this node frequently" or run builds on select nodes
- Ephemeral Agent : temporary use and destroy container/VM for a build.

StarNote

Jenkins : Jobs & Builds. Pg

No.	Schedule	Description	Time
1	Freestyle Projects	14	
2	Pipeline Projects	15	
3	Multibranch pipeline	16	
4	Build trigger	17	
5	Build parameters.	18	
6	SCM & Webhooks .	19	
7	Environment Variables	20	
8	Build history & Artifact	20	
9	[Advance] Syntax declarative	21–23	
10	[A] Stages.: Seq vs parallel	24	
11	[A] Error handling .	25	
12	.		
13			
14			
15			
16			
17			
18			
19			
20			
21			
22			
23			
24			
25			

StarNote

1. Freestyle project:

Simplest and most flexible type of Job.
helps us configure CI/CD job thru Jenkins UI by selecting options Source code mangm., build & triggers, build steps & post-build...

Different Config Settings:

a) General (contains basic information about the Job.)
   - Description: It describes what Job does
   - Discard Old build: Auto remove old builds.

b) Source Code Management: This tells Jenkins where the application source code is located.

c) Credentials: to access private repo, Jenkins needs credentials

d) Build triggers: determines when Jenkins should start a job
   i) Periodic Build ii) Poll SCM iii) GitHub Hook Trigger

e) Build Environment: config conditions/env setting

f) Build Steps: define what Jenkins actually does during the build.

StarNote

2. Pipeline Projects:

A pipeline project defines entire CI/CD process as code, written in a Groovy - based DSL.

It models builds, test and deploy as a sequence of Stages.

definition line in Jenkins UI or in a <Jenkinsfile> in Git repo.

Pipeline script: Code pasted directly into the job config.

Pipeline script from SCM: Jenkins pulls the <Jenkinsfile> from Git at build time.

2 type of Syntaxes:

Declarative	Scripted
Strict, predefined blocks.	Free - form Groovy.
pipeline { }	node { }
Moderate flexibility	High flexibility

StarNote

3. Multibranch Pipeline

MBP → • builds
• tests
• deploy

Source control repository

multiple branches.

Without requiring us to manually create Jenkins Job / branch

Workflow:

Bind Jenkins → Source Repo ✓ not branch ✗

Jenkins looks thru every branch in search of Jenkinsfile

if branch is deleted → Jenkins → Auto removes any build job <cleaning>

if found → Auto configures dedicated Job and executes pipeline

Benefits:

- Automation (Zero touch)
- Branch Isolation (0 main branch interfere)
- Version Controlled pipeline (Branch-wise CI/CD)

StarNote

④ Build Triggers

Build Triggers define when Jenkins automatically starts a build.

Common Trigger type:

1. Remote trigger : Start build by calling URL:

http://<jenkins>:8080/job/<job>/build?token=<TOKEN>

2. Build after other projects are built. runs when another job finishes. Used to chain Jobs.

3. Build periodically (cron)

Runs on a schedule regardless of code changes

4. Poll SCM (discussed Later)

5. GitHub WebHook trigger. (discussed Later)

CRON Syntax: Min Hour Date Month Day

0-59 0-23 1-31 1-12 0-7 (0 → Sunday)

(H) hash : spreads load by picking consistent but staggered minute, so that all build dont start at once.

StarNote

(5) BUILD PARAMETERS

Build parameters let a job accept input values at build time, so one job can behave differently per run without changing the config.

How to enable: freestyle: "this project is parameterized"

pipeline: "parameters" block

Types:

String: Single-line text

Text: Multi-line text

Boolean: checkbox (T/F)

Choice: Dropdown list of fixed opts.

password: Masked input

File: (upload)

Run: Pick a build of another job

Git parameter: choose a branch or tag from the repo.

StarNote

(6) SCM & Webhooks.

a) Source Code Management: System to store and manage source code and its change.

What Jenkins does is connect to your repository and get the lastest code to build.

b) Poll SCM: periodically checks SCM repository to see whether new code has been committed.

H/5 * * * * Every ~5min Jenkins checks on GitHub for latest code changes/commit.

if No changes → Do nothing
New commit → Start the build.

c) GitHub Webhook: Works the opposite way to poll SCM, here github checks for changes and notifies Webhook — Jenkins starts the build after receiving info GitHub.

Difference: Poll SCM: Slight delay, More Netreq starts after confirmn
Webhook: faster, less Netreq. Build starts immediately

(Webhook is more agile than SCM)

7. Environment Variable:

Environment variables are key-pair that store information which can be used by Jenkins jobs, pipeline, scripts and application during execution.

Temporary settings / information available to a build

Common Jenkins environment variable:

Build_Number; Build_ID; Job_Name; Workspace; Build_url; Node_Name; GIT_COMMIT.

8. Build history & Artifacts.

Build History is the list of all previous and current builds executed by a jenkins job.

Artifacts are the files produced by a build that Jenkins saves so they can be downloaded or used later.

→ Stored in [myapp.jar]

StarNote

Advanced portions. ⑨ Pipeline Syntax:

Jenkins file contains the entire CI/CD process in form of pipeline.

Components:

a. agent: tells where the pipeline should run.

```
agent any (Runs this pipeline on any available jenkins node)

agent { label 'linux' } Specify agent.
```

b. Stages: contains major phases of the pipeline.

```
Stages {
    stage('Build') {
        step {
            echo 'Building'
        }
    }
    ..... Same for test, deploy.
}
```

Extra. tools: tells jenkins which build tools pipeline needs.

StarNote

c. Step: contains the actual commands/action jenkins need to perform.

```
steps {
    echo 'Hello'
    sh 'mvn clean package'
}
```

d. Post: defins what actions Jenkins performs after a pipeline finishes.

5 common condition: always, success, unstable, aborted.

e. Environment: defines environment variable that can be used during pipeline.

f. parameter: allows to take input during build start

```
parameters { choice(
    name: 'Environment'
    choices: ['dev', ....]
    description: "msg"
)}
```

StarNote

g. when: decides whether to run a stage or skip.

```
stage('deploy') {
    when {
        branch 'main'
    }
}
```

```
when {
  expression {
    .....
  }
}
```

h. input: awaits human confirmation (approval/inp) before proceeding

```
Stage('Deploy') {
    steps{
        input message: '...'
    }
}
```

i. options: controls how jenkins execute and manage pipeline

to timestamps to console log:

```
options { timestamps() }
```

StarNote

(10) Build Stages. (Sequential Vs parallel)

Sequential Stages....

Sequential means stages execute one after another. in the order they are written.

what happens when a stage fails — Restore Shipped. Each stages depends on the output of previous build

- Simple, predictable, easy to debug
- Total time is the sum of all stages, so slower in overall process.

Parallel Stages

Independent Stages run at a same time inside a parallel block, and the parent Stage finishes when all branch finish!

Multiple independent — stages run Simultaneously

Tasks run without hampering other task. each branch can run on a different agent. Total time ≃ slowest branch.
logs are interleaved, hard to read

11. Error handling

Error handling means deciding what Jenkins performs when something doesn't work.

post { failure } : action performed when pipeline fails.

catchError() : allows us to catch an error and control the result instead of stopping immediately everything.

[loc: stages, step]

try/catch() : Same as catchError.

StarNote

Jenkins Plugins

No.	Schedule	Description	Time
1	plugins ?		
2	Plugin dependencies	27	
3	Common plugins		
4	pipeline plugins	28	
5	Credentials plugin	29	
6	Git/GitHub/Maven/Docker/		
7	Kubernetes plugins		
8			
9			
10			
11			
12			
13			
14			
15			
16			
17			
18			
19			
20			
21			
22			
23			
24			
25			

StarNote

1. Jenkins Plugins :

Jenkins plugin is an extension that add extra-features or integrations to Jenkins
↳ Adds capability to Jenkins

Jenkins provide CI/CD
+
plugins ↳ work with Git, Docker, Maven, kubernetes, etc.

2. Plugin Dependencies :

plugin dependencies means one Jenkins plugin depend another plugin to work properly.

Installing one plugin can automatically install its required dependencies.

3. Common Plugins :

Git : Jenkins → Git repos.
GitHub: Jenkins → GitHub
pipeline : CI/CD pipeline → using Jenkinsfile

BlueOcean: Visual interface for pipeline

Junit: Display automated test result.

StarNote

4. Pipeline Plugins:

Pipeline functionality is provided by a collection of plugins, rather than one single plugin.

like:

- pipeline: provides basic pipeline features/ functionalities.
  - pipeline: Stage View: Shows stages such as: Build → Test → Deploy.
  - pipeline: Basic Steps: provides common steps. echo, sh.
  - pipeline: groovy: Provides groovy scripting support used by Jenkins pipeline
  - pipeline: Input step: Allows Jenkins to pause and wait for manual approval.

Workflow:

Source → GitHub → Git/GitHub → Jenkins pipeline → Maven (Build) → JUnit (Display Test results) → Docker (Image) → Kubernetes (Deploy) → Slack Plugin (Notify).

5. Credentials Plugin

Allows Jenkins to securely store and manage sensitive information. such as password, SSH, key, API tokens, Certificate

Bad practice: Storing Credentials in Jenkinsfile (Hardcoding)

Good practice: Credentials → store → Jenkins Credential Manager

Credential ID ← Reference by

Concepts:

Credential ID: Name used to reference a stored credential

Secret Masking: Jenkins tries to hide secret in console output.

Scope: Controls where a credential can be used.

StarNote

Jenkins Configuration as Code plugin.

Jenkins Configuration as Code (JCasC) is a core feature that allow us to define entire Jenkin's controller's config → [as Code] by using human readable, plain-text YAML

What it eliminates:

Jenkins UI Config
Maintaining complex Groovy initialisation Scripts.

Benefits:

- Version Control: Track config updates, manage changes via pull req, easy roll back
- Disaster Recovery: Spin up identical controller from scratch in minutes if a Server crashes
  Eliminate
- 0 drift: Snowflake Server (uniquely managed Server that becomes fragile, difficult to reproduce).

StarNote

StarNote

Study Plan

Date : 33-34

Subjects :

- Credentials
- Username/password
- SSH keys
- Secret text

Subjects :

- Secret file
- Credential Binding
- RBAC
- User & permission

Subjects :

- Agent Security
- Securing Jenkins.
-
-

Subjects :

-
-
-
-

Performance

Credential & Security Jenkins.

Summary

Shot notes.

StarNote

Jenkins Credential Security Overview

Credentials: Securely stores password tokens, SSH keys, Certificates.

Username/password: Stores a username and password, commonly used for Git repositories, databases.

SSH keys: Stores an SSH private key for secure authentication to Git servers, linux machine.

Secret text: Stores a single secret value such as an API token, access token.

Secret file: Stores sensitive file such as configuration file, etc

Credentials Binding: Makes credential available to a pipeline temporarily as env. variable ..... without exposing value.

StarNote

RBAC (Role based Access Control)

Assigns permission based on roles. Usually implemented using an authorization plugin

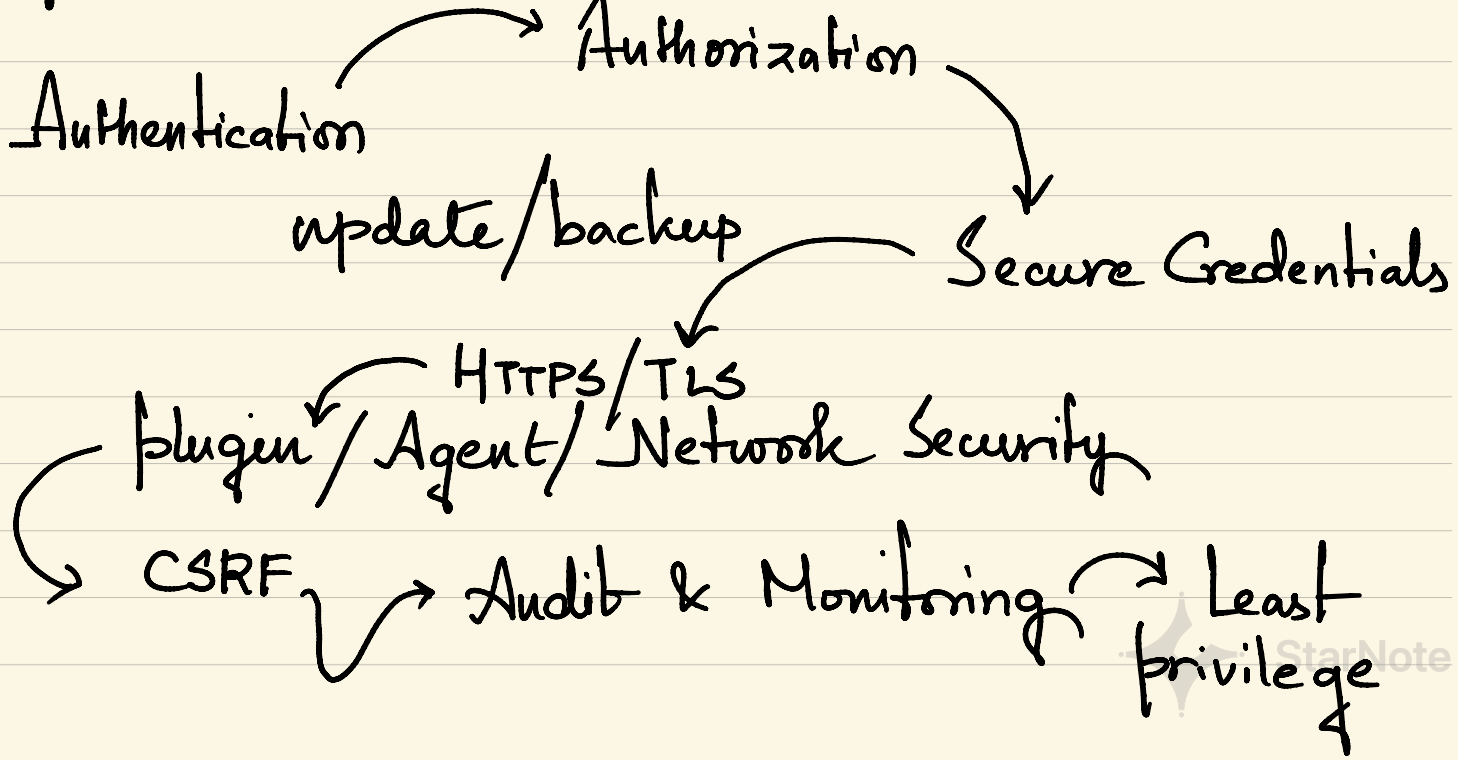
User & Permission: Controls who can log in and what they can do —

view jobs, build, Config, administer Jenkins

Agent Security: Protects Jenkins Agents by using Secure comms, proper permission and avoiding unnecessary access.

Securing Jenkins:

protecting the Jenkins Server, users, credentials job & agents.



StarNote

Controller & Agent

No.	Schedule	Description	Time
1	Controller vs Agent	36	
2	Static Agent	36	
3	Dynamic Agent	37	
4	SSH Agent	37	
5	Environment Conf.	38	
6	Docker Agent	38	
7	Distributed Agent Lifecy.	39	
8	JNLP	40	
9			
10			
11			
12			
13			
14			
15			
16			
17			
18			
19			
20			
21			
22			
23			
24			
25			

StarNote

1. Controller vs Agent

Controller	Agent
'Brain' of the operation.	'muscle' that performs task.
Manages Agents	Connects to the controller for Job.
Task: Hosts Jenkins web UI, stores config, schedules jobs, monitors agents and build results.	Executes a job when controller schedules it. Agent runs the code, compiles it, runs the test and sends the logs and final status back to Controller.

2. Static Agent

These are permanent machines (dedicated EC2, bare metal server .... etc) that are always on and always connected to the Jenkins controller

feature:

- fast to start a job
- Always running
- Costs even when idle.

StarNote

3. Dynamic Agents

Spun-up on demand just before a build starts, destroyed immediately after the build finishes.

Jenkins → connect → (Aws, Azure, kubernetes) → req new machine

runs the build

throws away after Job is done

- Cost Effective, Clean and ideal for every build.

4. SSH Agent: controller to agent connection.

(JC) → SSH connection → Remote Machine

- logs in using cred
- downloads agent.jar

starts the process

* Common way to connect linux-based static agent.

Labels: A tag assigned to an Agent to describe its capabilities. JC looks for available Agent with matching label and sends job there.

StarNote

5. Environment Configuration:

Agent Workspace: A specific directory on Agent's hard-drive where build takes place. When job runs,

Jenkins → checkout → Source code → Git → Workspace

create temporary files and compiles artifact. (AGW)

6. When to use docker Agents:

- Installing Java, Node.js, python and maven directly on Agent OS.

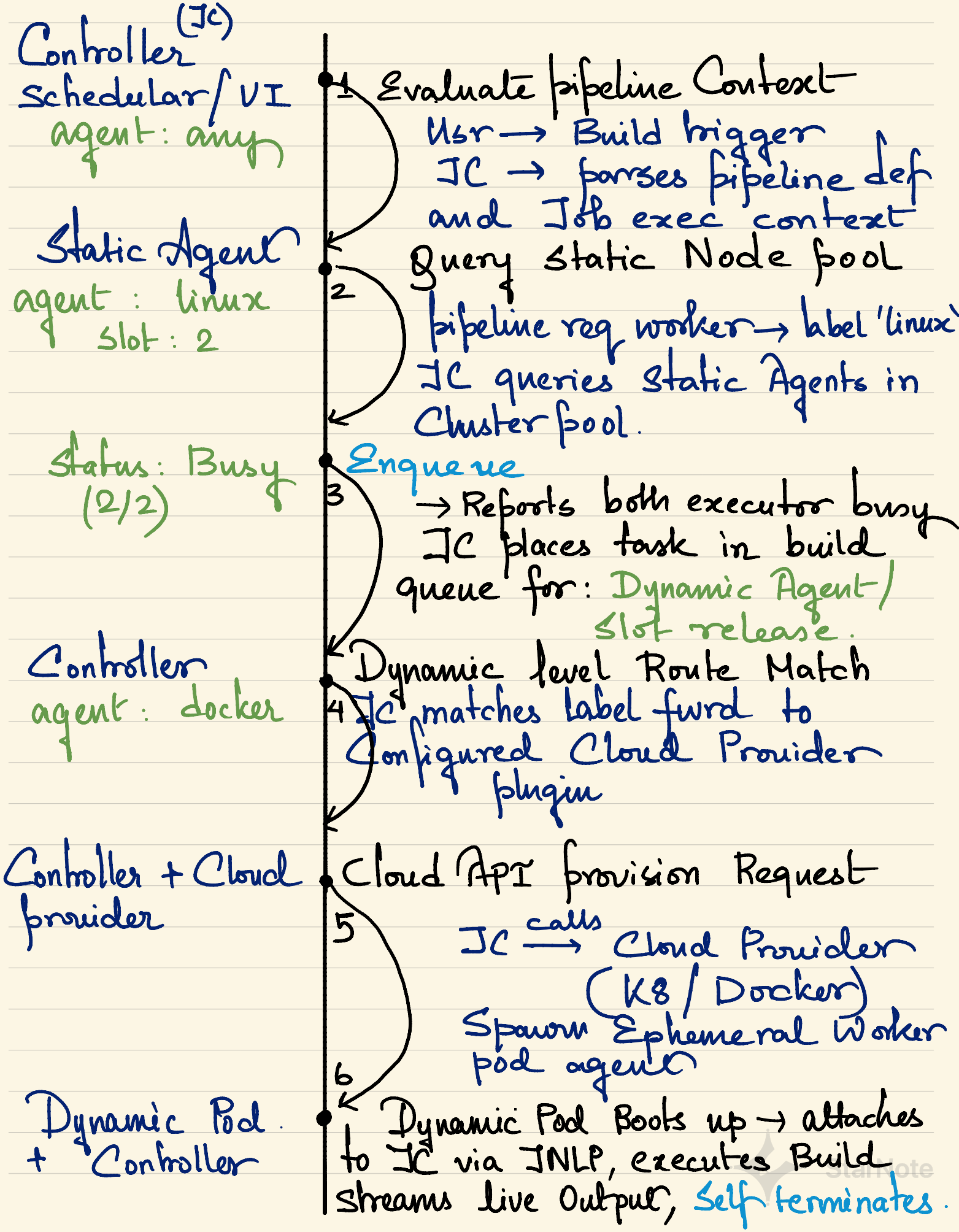
Instead Run:

Jenkins Agent runs pipeline inside Specific Docker Container

This guarantees build Environment always same, no need to manage Software versions on the underlying host machine.

StarNote

(7.) Distributed Agent lifecycle (39)



8. (Java Network launch Protocol) JNLP

way to connect Jenkins agent to Controller

Jenkins Agent —initiates JNLP→ JC.

useful when agent is behind firewall/NAT

JC ⟶✗ Agent directly.

Working

- Jenkins creates an Agent (or registers)
- Agent Starts and gets information (connection)
- Agent initiates connection to JC.
- Once online JC sends build task. to agent.

StarNote